

Linux Embebido – Módulo 3

Concurrencia en espacio de usuario

Ing. Marín Ariel · Fabrizio Carlassara (BIDOC) ·

Pablo Gómez (BIDOC) · Alexis Hernández de la Cruz (BIS)

Técnicas Digitales III

Laboratorio de Sistemas Embebidos

Universidad Tecnológica Nacional

Facultad Regional Avellaneda

Departamento de Ingeniería Electrónica

Índice

1. Objetivos del módulo	3
2. Procesos e hilos en Linux	3
2.1. De espacio de kernel a espacio de usuario	3
2.2. El proceso de Linux	4
2.3. Hilos: livianos y compartidos	4
2.4. Comparación con FreeRTOS	5
2.5. La API pthread: crear y esperar hilos	5
2.6. Herramientas para observar procesos e hilos	6
3. El problema de la concurrencia: condición de carrera	8
3.1. Sección crítica y pérdida de actualización	8
3.2. Mutex POSIX	8
3.3. Compilación con soporte de hilos	10
4. Variables de condición	10
4.1. El problema del busy-wait	10
4.2. La API pthread_cond_t	10
4.3. Spurious wakeups: por qué while y no if	11
5. Práctica 1: demostración de condición de carrera	11
5.1. Compilación y ejecución	12
6. Práctica 2: productor-consumidor con buffer circular	13
6.1. Diseño del buffer circular	13
6.2. El código	13
6.3. Flujo de sincronización	15
6.4. Compilación y ejecución	16
7. Políticas de scheduling con hilos	16
7.1. Recordatorio del Módulo 1	16
7.2. Atributos de scheduling en pthread_attr_t	17
7.3. Ejemplo: hilo de control con tiempo real	18
7.4. Prioridad de procesos con nice y renice	19
8. Cierre conceptual	20

Presentación del módulo

En los módulos anteriores se construyeron módulos que corren dentro del kernel. En este módulo se trabaja del otro lado de la frontera: el espacio de usuario. El objetivo central es programar aplicaciones concurrentes usando la API de hilos de POSIX (`pthread`). El concepto de ejecución paralela ya se vio en el contexto de FreeRTOS; aquí se establece el puente con el modelo de Linux, se introducen los mecanismos de sincronización (*mutex* y variables de condición) y se implementa el patrón clásico *productor-consumidor* con un buffer circular.

1. Objetivos del módulo

Al finalizar este módulo se debería poder:

- Distinguir proceso e hilo en Linux y explicar qué memoria comparten y cuál es privada para cada hilo.
- Crear hilos con `pthread_create` y sincronizar su terminación con `pthread_join`.
- Identificar una sección crítica y protegerla con un mutex.
- Usar variables de condición para coordinar hilos que producen y consumen datos de un buffer compartido.
- Implementar el patrón productor-consumidor con buffer circular en C.
- Asignar una política de *scheduling* de tiempo real a un hilo con `pthread_attr_setschedpolicy`.
- Observar procesos e hilos en ejecución con `ps`, `top` y el sistema de archivos `/proc`.

2. Procesos e hilos en Linux

2.1. De espacio de kernel a espacio de usuario

En el Módulo 2 todo el código corrió dentro del kernel: los módulos `.ko` se cargan en el mismo espacio de memoria que el propio kernel, tienen acceso directo al hardware y usan una API completamente distinta a la biblioteca estándar de C (no existe `printf`, no existen `malloc/free` tal como se conocen, no se puede bloquear libremente).

A partir de este módulo el foco cambia al **espacio de usuario**: programas ordinarios que corren con privilegio restringido, acceden al hardware solo a través de syscalls y pueden usar toda la biblioteca estándar de C.

La concurrencia en el kernel tiene reglas mucho más estrictas: hay contextos donde no se puede dormir (por ejemplo, dentro de un manejador de interrupciones), los *spinlocks* reemplazan a los mutex en muchos casos, y un error puede colgar el sistema entero. El kernel tiene su propia API de hilos (`kthread_create()`) para tareas de fondo como `kswapd` o `kworker`, que corren completamente en espacio de kernel. Este módulo se concentra en los hilos de espacio de usuario, donde los mismos conceptos aplican sin esas restricciones adicionales.

2.2. El proceso de Linux

Un **proceso** es la unidad de aislamiento en Linux. Cada proceso tiene:

- Un **espacio de memoria virtual propio**: código, datos globales, *heap* y pila de llamadas (*stack*), todos en un espacio de direcciones separado del resto de los procesos.
- Un **identificador** (PID) y una tabla de descriptores de archivo propia (archivos abiertos, sockets, *pipes*).
- Su propio contexto de señales, uid/gid efectivo y límites de recursos.

Para crear un proceso nuevo se usa `fork()`, que genera una copia del proceso padre. Para crear hilos dentro del mismo proceso se usa `pthread_create()`, que internamente utiliza la syscall `clone()` del kernel con flags que indican qué recursos compartir.

2.3. Hilos: livianos y compartidos

Un **hilo** (*thread*) es una unidad de ejecución dentro de un proceso. Todos los hilos de un mismo proceso:

- **Comparten**: espacio de código, *heap*, variables globales y descriptores de archivo.
- **Tienen propio**: pila de llamadas, registros del procesador (PC, SP, etc.) y máscara de señales.

Crear un hilo es mucho más barato que crear un proceso nuevo: no hay copia de espacio de memoria, y la comunicación entre hilos del mismo proceso se hace a través de variables compartidas sin necesidad de mecanismos de IPC.

En Linux, los hilos se implementan mediante la biblioteca NPTL (*Native POSIX Thread Library*). Desde el punto de vista del kernel son “procesos livianos” (LWP), visibles con `ps -eLf`, pero comparten el mismo espacio de memoria.

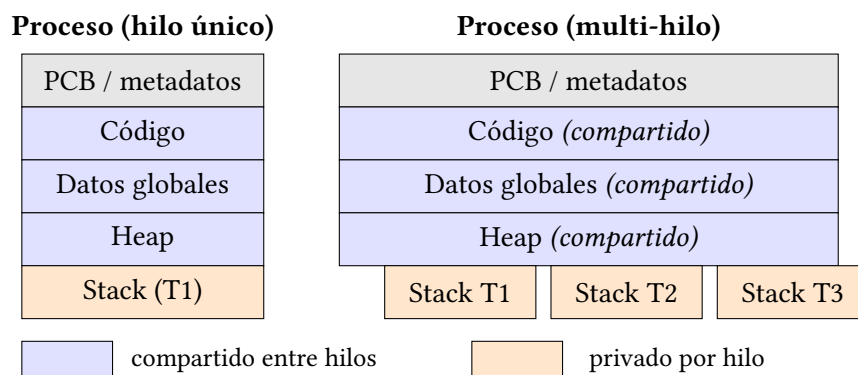


Figura 1: Comparación entre un proceso con un único hilo (izquierda) y un proceso con tres hilos (derecha). El código, los datos globales y el heap son compartidos; cada hilo tiene su propia pila de llamadas.

2.4. Comparación con FreeRTOS

El modelo de concurrencia de FreeRTOS ya fue introducido en el bloque de RTOS. La Tabla 1 establece la correspondencia con la API de hilos de Linux.

Concepto	FreeRTOS	Linux
Unidad de ejecución	xTaskCreate()	pthread_create()
Espacio de memoria	Todos comparten el mismo espacio	Comparten dentro del proceso
Scheduling	Basado en prioridades (cooperativo o expropiativo)	CFS o políticas RT
Mutex	xSemaphoreCreateMutex()	pthread_mutex_t
Semáforo	xSemaphoreCreateBinary()	sem_t
Señalización	xSemaphoreGive()	pthread_cond_signal()
Cola de mensajes	xQueueCreate()	Implementar con mutex + cond var
Suspender	vTaskDelay()	usleep() / nanosleep()

Cuadro 1: Correspondencia entre conceptos de FreeRTOS y la API de hilos de Linux.

La diferencia conceptual más importante es que en FreeRTOS todas las tareas comparten el mismo espacio de memoria (hay un único espacio de direcciones en el microcontrolador), mientras que en Linux cada proceso tiene su propio espacio y los hilos solo comparten dentro de ese proceso. Esto hace que un error de escritura en un hilo pueda corromper datos de otro hilo del mismo proceso, pero *no* puede afectar a otros procesos.

2.5. La API pthread: crear y esperar hilos

Las funciones fundamentales para trabajar con hilos son:

```
#include <pthread.h>

/* Crear un hilo. Devuelve 0 si tuvo exito, codigo de error si fallo */
int pthread_create(pthread_t *thread,
    const pthread_attr_t *attr, /* NULL = atributos por defecto */
    void *(*start_routine)(void *), /* funcion del hilo */
    void *arg); /* argumento de la funcion */

/* Esperar a que un hilo termine. Captura el valor de retorno. */
int pthread_join(pthread_t thread, void **retval);

/* Terminar el hilo actual y devolver un valor */
void pthread_exit(void *retval);
```

La función que ejecuta el hilo tiene la firma `void *func(void *arg)`: recibe un puntero genérico como argumento y devuelve un puntero genérico. Para pasar o devolver estructuras más complejas, se usa un `struct` y se hace *cast* del puntero.

```
#include <stdio.h>
#include <pthread.h>

static void *saludar(void *arg)
{
    int num = *(int *)arg;
    printf("Hilo %d en ejecucion\n", num);
    return NULL;
}

int main(void)
{
    pthread_t t1, t2;
    int n1 = 1, n2 = 2;

    pthread_create(&t1, NULL, saludar, &n1);
    pthread_create(&t2, NULL, saludar, &n2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Ambos hilos terminaron\n");
    return 0;
}
```

Nota

No hay garantía de en qué orden ejecutarán los hilos. La salida puede ser “Hilo 1” antes que “Hilo 2” o al revés, dependiendo del scheduler. Esta impredecibilidad es el punto de partida de toda la problemática de concurrencia.

2.6. Herramientas para observar procesos e hilos

Una vez que un programa con múltiples hilos está corriendo, hay varias formas de observar su estado desde otra terminal. Los ejemplos siguientes usan el programa `producer_consumer` desarrollado más adelante en este módulo.

ps: listar procesos y hilos

```
# Listar todos los procesos con informacion basica
ps aux
```

```
# Listar procesos Y sus hilos (columna LWP = Light Weight Process = hilo)
ps -eLf | grep producer_consumer
```

Para un proceso con 3 hilos (main + productor + consumidor), `ps -eLf` muestra 3 filas con el mismo PID pero distintos LWP:

```
UID PID PPID LWP C NLWP STIME TTY TIME CMD
lse 1234 1100 1234 0 3 10:00 pts/0 0:00 ./producer_consumer
lse 1234 1100 1235 0 3 10:00 pts/0 0:00 ./producer_consumer
lse 1234 1100 1236 0 3 10:00 pts/0 0:00 ./producer_consumer
```

Columna	Significado
UID	Usuario propietario del proceso
PID	Identificador del proceso (compartido por todos sus hilos)
PPID	PID del proceso padre
LWP	Identificador del hilo (<i>Light Weight Process</i>); distinto por hilo
C	Uso de CPU en el último instante de muestreo (porcentaje entero)
NLWP	Número total de hilos del proceso
STIME	Hora de inicio del proceso
TTY	Terminal asociada (pts/0 = pseudoterminal; ? = sin terminal)
TIME	Tiempo de CPU acumulado desde el inicio
CMD	Comando que inició el proceso

Cuadro 2: Columnas de `ps -eLf`.

top con vista de hilos

```
# Mostrar hilos individuales en top (equivale a presionar 'H' dentro de
  top)
top -H

# Filtrar por PID específico
top -H -p $(pgrep producer_consumer)
```

Con `-H`, cada fila representa un hilo en lugar de un proceso. La columna %CPU permite ver cuánto CPU consume cada hilo individualmente.

/proc: el sistema de archivos del kernel

El directorio `/proc/<pid>/` expone el estado interno del proceso directamente desde el kernel:

```
# Obtener el PID del programa
PID=$(pgrep producer_consumer)

# Ver cantidad de hilos y estado general del proceso
```

```
cat /proc/$PID/status | grep -E "(Name|Pid|Threads|VmRSS)"

# Directorio con un subdirectorio por cada hilo del proceso
ls /proc/$PID/task/

# Ver el estado de scheduling de un hilo específico
cat /proc/$PID/task/$PID/sched
```

Salida típica de `/proc/$PID/status`:

```
Name: producer_consum
Pid: 1234
Threads: 3
VmRSS: 1024 kB
```

`Threads: 3` confirma que el proceso tiene 3 hilos en ejecución. `VmRSS` es la memoria residente (cuánta RAM física usa el proceso en este momento).

pstree: árbol de procesos

```
# Mostrar el árbol de procesos con PIDs
pstree -p

# Mostrar solo el subárbol del programa
pstree -p $(pgrep producer_consumer)
```

3. El problema de la concurrencia: condición de carrera

3.1. Sección crítica y pérdida de actualización

Cuando dos hilos acceden a un dato compartido y al menos uno de ellos escribe, el resultado puede depender del orden exacto de ejecución. A esta situación se la llama **condición de carrera** (*race condition*).

El ejemplo más sencillo es un contador compartido. Aunque en C se escribe como una sola línea (`counter++`), el procesador lo ejecuta como **tres instrucciones**: `LOAD` (leer de memoria al registro), `ADD` (incrementar en el registro) y `STORE` (escribir de vuelta a memoria). Si el scheduler interrumpe a T1 entre estas instrucciones y le da CPU a T2, se puede perder una actualización:

Una **sección crítica** es el fragmento de código que accede a datos compartidos y que debe ejecutarse de forma **atómica** (sin interrupciones de otros hilos). Para garantizar esa atomicidad se usan mecanismos de exclusión mutua.

3.2. Mutex POSIX

Un **mutex** (*mutual exclusion*) es una primitiva de sincronización que garantiza que solo un hilo a la vez pueda estar dentro de la sección crítica. Si T2 intenta tomar un mutex que T1 ya tomó, T2 queda bloqueado (suspendido por el kernel, sin consumir CPU) hasta que T1 lo libere.

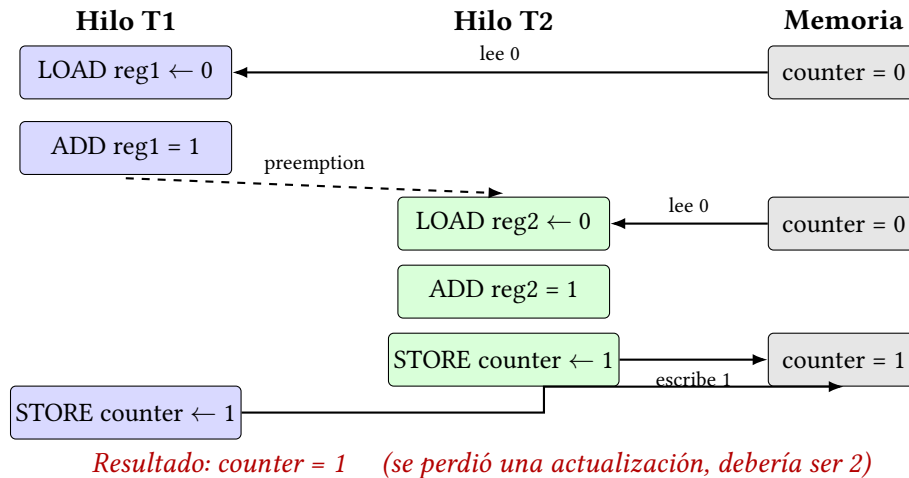


Figura 2: Condición de carrera en un contador compartido. T1 lee el valor 0 y es interrumpido antes de escribir. T2 también lee 0, incrementa y escribe 1. Cuando T1 retoma la ejecución, también escribe 1. El resultado final es 1 en lugar de 2.

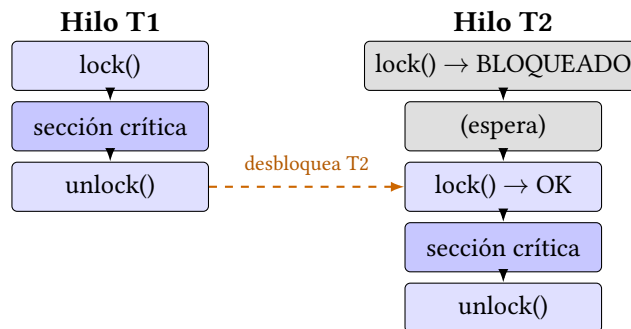


Figura 3: Mutex en acción. T2 intenta tomar el mutex mientras T1 lo tiene; queda bloqueado hasta que T1 llama `unlock()`. El kernel retoma T2 sin que se desperdicie CPU en espera activa.

La API de mutex POSIX es:

```
#include <pthread.h>

/* Inicializacion estatica (mas comun en variables globales) */
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

/* Inicializacion dinamica (cuando el mutex esta en un struct o heap) */
pthread_mutex_init(&mutex, NULL); /* NULL = atributos por defecto */

/* Tomar el mutex (bloquea si ya esta tomado) */
pthread_mutex_lock(&mutex);

/* --- seccion critica --- */

/* Liberar el mutex */
pthread_mutex_unlock(&mutex);
```

```
/* Liberar los recursos (para mutex inicializado dinamicamente) */
pthread_mutex_destroy(&mutex);
```

Regla fundamental: **todo acceso a datos compartidos entre hilos debe estar protegido por el mismo mutex**. Si T1 protege la escritura pero T2 lee sin tomar el mutex, la condición de carrera sigue existiendo.

3.3. Compilación con soporte de hilos

Los programas que usan pthreads deben linkear contra la biblioteca correspondiente. La forma recomendada es usar el flag `-pthread` (en lugar de solo `-lpthread`), que además ajusta algunas definiciones del preprocesador:

```
gcc -Wall -pthread -o programa programa.c
```

4. Variables de condición

4.1. El problema del busy-wait

Con un mutex se puede proteger el acceso a datos compartidos, pero aparece un problema cuando un hilo necesita *esperar* a que se cumpla una condición. Por ejemplo, un consumidor que quiere leer de un buffer vacío. Sin variables de condición, las únicas opciones son:

- **Busy-wait:** tomar el mutex, verificar si hay datos, liberarlo, repetir. Consume CPU al 100 % mientras espera.
- **Polling con sleep:** tomar el mutex, verificar, soltar, dormir un tiempo fijo, repetir. Desperdicia latencia y es frágil.

Ninguna de las dos es aceptable en un sistema embebido donde la CPU puede estar haciendo trabajo útil mientras el buffer está vacío.

4.2. La API `pthread_cond_t`

Una **variable de condición** permite a un hilo **bloquearse eficientemente** hasta que otro hilo le señale que la condición que esperaba ya se cumplió. Siempre se usa junto con un mutex: la variable de condición señala el cambio de estado; el mutex protege la variable de estado en sí.

```
#include <pthread.h>

/* Inicializacion */
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
/* o dinamicamente: pthread_cond_init(&cond, NULL); */

/* Esperar (llamar con el mutex ya tomado):
1. libera el mutex atomicamente
```

```

    2. bloquea el hilo
    3. cuando es despertado, vuelve a tomar el mutex */
pthread_cond_wait(&cond, &mutex);

/* Despertar uno de los hilos bloqueados en esta cond var */
pthread_cond_signal(&cond);

/* Despertar todos los hilos bloqueados en esta cond var */
pthread_cond_broadcast(&cond);

/* Liberar recursos */
pthread_cond_destroy(&cond);

```

El punto clave de `pthread_cond_wait` es que libera el mutex y bloquea el hilo **de forma atómica**: no existe un instante en que el mutex esté suelto pero el hilo todavía no esté bloqueado, lo que eliminaría la señal.

4.3. Spurious wakeups: por qué **while** y no **if**

POSIX permite que `pthread_cond_wait` regrese aunque nadie haya llamado `signal()` o `broadcast()`. Esto se llama **spurious wakeup** (despertar espurio) y puede ocurrir en ciertas implementaciones del sistema operativo como efecto de señales u otras interrupciones internas. Además, incluso cuando sí hubo un `signal()`, es posible que otro hilo haya consumido la condición antes de que el hilo despertado tenga oportunidad de ejecutarse.

Por esto, la verificación de la condición de espera **siempre debe hacerse en un `while`**, nunca en un `if`:

```

/* CORRECTO: re-verifica la condicion al despertar */
pthread_mutex_lock(&mutex);
while (!hay_datos)
    pthread_cond_wait(&cond, &mutex);
/* aqui se garantiza que hay_datos es true */
pthread_mutex_unlock(&mutex);

/* INCORRECTO: no re-verifica */
pthread_mutex_lock(&mutex);
if (!hay_datos)
    pthread_cond_wait(&cond, &mutex);
/* aqui NO se garantiza nada */
pthread_mutex_unlock(&mutex);

```

5. Práctica 1: demostración de condición de carrera

Este programa demuestra el efecto de una condición de carrera en un contador compartido. Dos hilos incrementan el mismo contador `ITERACIONES` veces cada uno. Sin mutex, el resultado difiere del esperado; con mutex, es siempre correcto.

```
/* race_demo.c */
#include <stdio.h>
#include <pthread.h>

#define ITERACIONES 1000000

static long counter = 0;
static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

static void *incrementar(void *arg)
{
    for (int i = 0; i < ITERACIONES; i++) {
        /* Descomentar las dos líneas siguientes para activar la proteccion
        */
        /* pthread_mutex_lock(&mutex); */
        counter++;
        /* pthread_mutex_unlock(&mutex); */
    }
    return NULL;
}

int main(void)
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, incrementar, NULL);
    pthread_create(&t2, NULL, incrementar, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Esperado: %d\n", 2 * ITERACIONES);
    printf("Obtenido: %ld\n", counter);
    return 0;
}
```

5.1. Compilación y ejecución

```
gcc -Wall -pthread -o race_demo race_demo.c
./race_demo
```

Resultado esperado sin mutex:

```
Esperado: 2000000
Obtenido: 1243872 # valor incorrecto y no reproducible
```

El resultado varía entre ejecuciones porque depende exactamente de cuándo el scheduler interrumpe cada hilo. Descomentando las dos líneas de mutex el resultado siempre es 2000000.

Nota

En algunos procesadores o con ciertas opciones del compilador, la condición de carrera puede no manifestarse si `counter++` se compila a una instrucción atómica (por ejemplo, `INC` en x86 con un único procesador). En la Raspberry Pi, con el compilador estándar y múltiples núcleos, el efecto es reproducible.

6. Práctica 2: productor-consumidor con buffer circular

El patrón **productor-consumidor** es la aplicación clásica de variables de condición. Un hilo *productor* genera datos y los deposita en un buffer compartido; un hilo *consumidor* retira datos del buffer y los procesa. El buffer actúa como colchón que absorbe diferencias de velocidad entre ambos.

6.1. Diseño del buffer circular

Se usa un **buffer circular** (también llamado *ring buffer*) de N slots. Dos índices avanzan de forma circular sobre el arreglo: `head` apunta al próximo slot libre donde el productor escribe; `tail` apunta al próximo slot con dato para que el consumidor lea. Un contador `count` registra cuántos ítems hay en el buffer en cada momento.

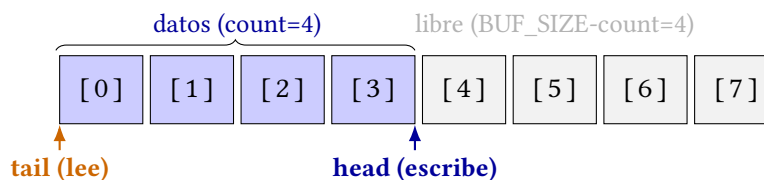


Figura 4: Buffer circular de 8 slots con 4 ítems. `tail` apunta al ítem más antiguo (el próximo a consumir); `head` apunta al slot libre donde el productor escribirá. Cuando `count == BUF_SIZE` el buffer está lleno; cuando `count == 0`, vacío.

Las condiciones de sincronización que gobiernan el buffer son:

- El productor debe **esperar** si el buffer está lleno (`count == BUF_SIZE`).
- El consumidor debe **esperar** si el buffer está vacío (`count == 0`).

Para modelar esto se usan **dos variables de condición**: `no_lleno` (el productor espera aquí cuando el buffer está lleno; el consumidor señala cuando consume) y `no_vacio` (el consumidor espera aquí cuando el buffer está vacío; el productor señala cuando produce).

6.2. El código

```
/* producer_consumer.c */
#include <stdio.h>
#include <pthread.h>
```

```

#include <unistd.h>
#include <time.h>

#define BUF_SIZE 8
#define N_ITEMS 20

/* --- Estructura del item --- */
typedef struct {
    int id;
    struct timespec ts; /* timestamp de produccion */
} item_t;

/* --- Buffer compartido --- */
static item_t buffer[BUF_SIZE];
static int head = 0;
static int tail = 0;
static int count = 0;

/* --- Primitivas de sincronizacion --- */
static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t no_lleno = PTHREAD_COND_INITIALIZER;
static pthread_cond_t no_vacio = PTHREAD_COND_INITIALIZER;

/* --- Hilo productor --- */
static void *productor(void *arg)
{
    for (int i = 0; i < N_ITEMS; i++) {
        item_t item;
        item.id = i;
        clock_gettime(CLOCK_MONOTONIC, &item.ts);

        pthread_mutex_lock(&mutex);

        /* Esperar si el buffer esta lleno */
        while (count == BUF_SIZE)
            pthread_cond_wait(&no_lleno, &mutex);

        /* Depositar el item */
        buffer[head] = item;
        head = (head + 1) % BUF_SIZE;
        count++;

        printf("[prod] item %2d producido | en buffer: %d\n", item.id,
            count);
        pthread_cond_signal(&no_vacio);
        pthread_mutex_unlock(&mutex);

        usleep(50000); /* 50 ms entre producciones */
    }
}

```

```

    }
    return NULL;
}

/* --- Hilo consumidor --- */
static void *consumidor(void *arg)
{
    for (int i = 0; i < N_ITEMS; i++) {
        pthread_mutex_lock(&mutex);

        /* Esperar si el buffer esta vacio */
        while (count == 0)
            pthread_cond_wait(&no_vacio, &mutex);

        /* Retirar el item */
        item_t item = buffer[tail];
        tail = (tail + 1) % BUF_SIZE;
        count--;

        printf("[cons] item %2d consumido | en buffer: %d\n", item.id,
            count);
        pthread_cond_signal(&no_lleno);
        pthread_mutex_unlock(&mutex);

        usleep(120000); /* 120 ms procesando (mas lento que el productor) */
    }
    return NULL;
}

int main(void)
{
    pthread_t prod, cons;

    pthread_create(&prod, NULL, productor, NULL);
    pthread_create(&cons, NULL, consumidor, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    return 0;
}

```

6.3. Flujo de sincronización

La figura anterior muestra el escenario donde el consumidor es más rápido que el productor y llega al buffer vacío. El camino inverso (productor bloqueado en `wait (no_lleno)` porque el buffer está lleno) es simétrico.

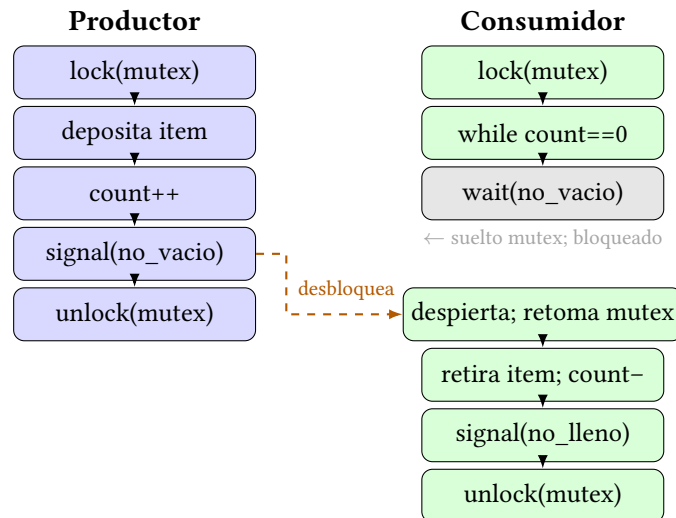


Figura 5: Flujo de sincronización cuando el consumidor llega al buffer vacío. El consumidor llama `wait`, libera el mutex y queda bloqueado. Cuando el productor deposita un ítem y llama `signal`, el consumidor se despierta, retoma el mutex y consume el ítem.

6.4. Compilación y ejecución

```
gcc -Wall -pthread -o producer_consumer producer_consumer.c
./producer_consumer
```

Salida esperada (fragmento):

```
[prod] item 0 producido | en buffer: 1
[cons] item 0 consumido | en buffer: 0
[prod] item 1 producido | en buffer: 1
[prod] item 2 producido | en buffer: 2
[prod] item 3 producido | en buffer: 3
[cons] item 1 consumido | en buffer: 2
[prod] item 4 producido | en buffer: 3
...
```

El productor (50 ms entre ítems) es más rápido que el consumidor (120 ms por ítem), por lo que el buffer se va llenando gradualmente. Cuando llega a `BUF_SIZE = 8`, el productor se bloquea en `wait(no_lleno)` hasta que el consumidor retira al menos un ítem.

7. Políticas de scheduling con hilos

7.1. Recordatorio del Módulo 1

En el Módulo 1 se vieron las políticas de scheduling de Linux. Las más relevantes para aplicaciones con requisitos de tiempo real son:

- **SCHED_OTHER**: el scheduler CFS por defecto. Busca equidad entre procesos, no determina plazos máximos de respuesta.

- **SCHED_FIFO**: tiempo real. Un hilo con esta política no es desalojado por hilos de menor prioridad; cede la CPU solo cuando se bloquea o un hilo de *mayor* prioridad queda listo para ejecutar.
- **SCHED_RR**: igual que SCHED_FIFO pero con quantum de tiempo: si hay varios hilos de la misma prioridad, se turnan.

Las prioridades de SCHED_FIFO y SCHED_RR van de 1 (mínima) a 99 (máxima). Un hilo con prioridad 90 siempre desaloja a uno con prioridad 70, sin importar cuánto tiempo de CPU haya usado.

7.2. Atributos de scheduling en `pthread_attr_t`

Para crear un hilo con una política de scheduling específica se usa `pthread_attr_t`:

```
#include <pthread.h>
#include <sched.h>

pthread_attr_t attr;
struct sched_param param;

pthread_attr_init(&attr);

/* Establecer la politica */
pthread_attr_setschedpolicy(&attr, SCHED_FIFO);

/* Establecer la prioridad */
param.sched_priority = 50;
pthread_attr_setschedparam(&attr, &param);

/* Sin esta linea, el hilo hereda la politica del hilo padre (SCHED_OTHER
)
ignorando los atributos que se acaban de configurar */
pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED);

/* Crear el hilo con estos atributos */
pthread_create(&tid, &attr, mi_funcion, NULL);

/* Liberar los atributos (el hilo ya fue creado, esto no lo afecta) */
pthread_attr_destroy(&attr);
```

Nota

Las políticas SCHED_FIFO y SCHED_RR requieren privilegios de superusuario o la capability CAP_SYS_NICE. En la Raspberry Pi, ejecutar con `sudo` es suficiente. Sin los privilegios necesarios, `pthread_create` devuelve EPERM.

7.3. Ejemplo: hilo de control con tiempo real

El siguiente ejemplo crea dos hilos: uno con SCHED_FIFO a prioridad 80 (hilo de “control”) y otro con SCHED_OTHER (hilo de “registro”). El hilo de control imprime su política y prioridad efectiva usando `sched_getscheduler` y `sched_getparam`.

```
/* rt_threads.c */
#include <stdio.h>
#include <pthread.h>
#include <sched.h>
#include <unistd.h>

static void imprimir_sched(const char *nombre)
{
    int policy = sched_getscheduler(0); /* 0 = hilo actual */
    struct sched_param p;
    sched_getparam(0, &p);

    const char *nombres_policy[] = {
        [SCHED_OTHER] = "SCHED_OTHER",
        [SCHED_FIFO] = "SCHED_FIFO",
        [SCHED_RR] = "SCHED_RR",
    };
    printf("[%s] politica: %s prioridad: %d\n",
        nombre, nombres_policy[policy], p.sched_priority);
}

static void *hilo_control(void *arg)
{
    imprimir_sched("control");
    /* En una aplicacion real: leer sensores, actuar sobre hardware */
    sleep(1);
    return NULL;
}

static void *hilo_registro(void *arg)
{
    imprimir_sched("registro");
    /* En una aplicacion real: escribir logs a disco */
    sleep(1);
    return NULL;
}

static pthread_t crear_hilo_rt(void *(*func)(void *), int prio)
{
    pthread_t tid;
    pthread_attr_t attr;
    struct sched_param param;
```

```

pthread_attr_init(&attr);
pthread_attr_setschedpolicy(&attr, SCHED_FIFO);
param.sched_priority = prio;
pthread_attr_setschedparam(&attr, &param);
pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED);
pthread_create(&tid, &attr, func, NULL);
pthread_attr_destroy(&attr);
return tid;
}

int main(void)
{
    pthread_t t_ctrl, t_log;

    /* Hilo de control: SCHED_FIFO, prioridad 80 */
    t_ctrl = crear_hilo_rt(hilo_control, 80);

    /* Hilo de registro: hereda SCHED_OTHER del main */
    pthread_create(&t_log, NULL, hilo_registro, NULL);

    pthread_join(t_ctrl, NULL);
    pthread_join(t_log, NULL);
    return 0;
}

```

```

# Requiere privilegios para SCHED_FIFO
gcc -Wall -pthread -o rt_threads rt_threads.c
sudo ./rt_threads

```

Salida esperada:

```

[control] politica: SCHED_FIFO prioridad: 80
[registro] politica: SCHED_OTHER prioridad: 0

```

Nota

En este ejemplo los dos hilos hacen `sleep(1)` y no compiten activamente por la CPU, por lo que el efecto de la prioridad no es observable directamente. En una aplicación real donde el hilo de control hace cálculos continuos (por ejemplo, un controlador PID ejecutando en un bucle cerrado), `SCHED_FIFO` garantiza que ningún hilo de menor prioridad lo interrumpa, lo que reduce la variación en el período de ejecución (*jitter*).

7.4. Prioridad de procesos con `nice` y `renice`

Para procesos que usan `SCHED_OTHER` (el scheduler CFS por defecto), la prioridad relativa se controla con el valor **nice**. El rango va de `-20` (mayor prioridad, más agresivo en pedir CPU) a `+19` (menor prioridad). El valor por defecto es `0`.

Solo el superusuario puede *disminuir* el nice (aumentar la prioridad); cualquier usuario puede aumentarlo (ceder CPU a otros procesos).

Al lanzar un proceso: **nice**

```
# Lanzar con nice +10 (menor prioridad que lo normal)
nice -n 10 ./producer_consumer

# Lanzar con nice -5 (mayor prioridad; requiere sudo)
sudo nice -n -5 ./producer_consumer
```

Cambiar el nice de un proceso en ejecución: **renice**

```
# Ver el nice actual de un proceso
ps -o pid,ni,comm -p $(pgrep producer_consumer)

# Cambiar el nice del proceso con PID 1234 a +15
renice -n 15 -p 1234

# Cambiar el nice de todos los procesos del usuario actual a +5
renice -n 5 -u $USER
```

Desde código C: la syscall **nice()**

```
#include <unistd.h>

/* Incrementar el nice en 10 (ceder prioridad) */
nice(10);

/* Para valores negativos se necesita CAP_SYS_NICE (root) */
nice(-5);
```

Nota

nice/renice solo afectan la prioridad *relativa* dentro de **SCHED_OTHER**: el CFS intenta dar equidad, pero un proceso con nice -20 recibirá significativamente más tiempo de CPU que uno con nice $+19$. Para garantías de tiempo real (latencia máxima acotada) se necesita **SCHED_FIFO** o **SCHED_RR**, como se vio en las secciones anteriores.

8. Cierre conceptual

Este módulo introdujo el modelo de concurrencia de Linux en espacio de usuario y los mecanismos que lo hacen seguro. Los puntos clave para retener:

- **Proceso vs. hilo:** el proceso es la unidad de aislamiento; el hilo es la unidad de ejecución. Los hilos de un proceso comparten heap y variables globales, lo que permite comunicación eficiente pero exige sincronización.
- **Mutex:** garantiza que solo un hilo a la vez ejecute la sección crítica. Siempre que dos hilos compartan datos mutables, el acceso debe estar protegido con el mismo mutex.
- **Variable de condición:** permite que un hilo espere eficientemente hasta que se cumpla una condición, sin consumir CPU. La verificación siempre va en un `while` para protegerse de spurious wakeups.
- **Productor-consumidor:** patrón fundamental en sistemas embebidos. Aparece en cualquier aplicación donde un hilo captura datos (sensor, UART) y otro los procesa.
- **Scheduling:** las políticas `SCHED_FIFO` y `SCHED_RR` permiten garantizar que un hilo crítico tenga acceso a la CPU antes que el resto, lo que reduce el *jitter* en bucles de control.

Referencias

- [1] Vasquez, F. y Simmonds, C. (2021). *Mastering Embedded Linux Programming*, 3.^a edición. Packt Publishing.
 - Cap. 17, *Processes and Threads* – modelo de procesos e hilos en Linux, API de `pthread`, mutex, variables de condición, semáforos POSIX y políticas de scheduling con `pthread_attr_t`.
- [2] Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press. Referencia exhaustiva de la programación de sistemas Linux. Los capítulos 29 a 33 cubren en profundidad la API POSIX de hilos: creación, sincronización, atributos, cancelación y seguridad en el uso de señales con hilos. Disponible también como referencia en línea en <https://man7.org/tlpi/>.
- [3] The Linux man-pages project. *pthread(7) – POSIX threads*. <https://man7.org/linux/man-pages/man7/pthreads.7.html>. Página de manual que describe el modelo de hilos POSIX en Linux (NPTL), compatibilidad con el estándar y comportamientos específicos de la implementación, incluyendo los spurious wakeups y la semántica de `pthread_cond_wait`.
- [4] The Linux man-pages project. *pthread_create(3)*. https://man7.org/linux/man-pages/man3/pthread_create.3.html. Documentación de la función de creación de hilos: firma, atributos, códigos de error y comportamiento en relación con el scheduler.
- [5] The Linux man-pages project. *pthread_mutex_init(3)*. https://man7.org/linux/man-pages/man3/pthread_mutex_init.3.html. Referencia completa de la

API de mutex POSIX: tipos de mutex, atributos, comportamiento ante intentos de doble lock y protocolos de inversión de prioridad.

- [6] The Linux man-pages project. *pthread_cond_init(3)*. https://man7.org/linux/man-pages/man3/pthread_cond_init.3.html. Referencia de variables de condición: semántica de wait, signal y broadcast, y la razón por la que el estándar POSIX permite spurious wakeups.
- [7] IEEE/The Open Group. *POSIX.1-2017 (IEEE Std 1003.1-2017)*. <https://pubs.opengroup.org/onlinepubs/9699919799/>. Estándar que define la API de hilos POSIX, incluyendo pthreads, mutex, variables de condición, semáforos y políticas de scheduling de tiempo real.